Hazelcast - MapReduce II

MapReduce - Etapas (Oculta)

El modelo al pasar de lo teórico a lo práctico tiene varias etapas más que las mencionadas:

- **Una etapa preliminar** donde se carga la información y se dispone la información para el framework
- **Una etapa intermedia** *post-mapper* llamada **`Combiner`** donde se sumarizan los datos
- **Una etapa final** de *post*-procesado que permite dar una última transformación al total de la información (tal vez ordenar o guardar en algún medio)

Las dos últimas etapas son opcionales

Key Predicate

Hazelcast - Key Predicate

`KeyPredicate` se utiliza para **pre-seleccionar los valores** que serán procesados

Cuenta con un método `evaluate` que recibe la clave y, **en caso de devolver `false`, el par no se procesa**

Útil para por ejemplo seleccionar un rango de datos

```
public interface KeyPredicate<Key> extends Serializable {

   boolean evaluate(Key key);

}
```

En el caso de que el `KeyValueSource` implemente el método `getAllKeys`, Hazelcast puede filtrar las claves antes de dividirlas en los nodos del cluster

Hazelcast - Key Predicate

Para el **WordCount** implementado:

- Si queremos considerar **únicamente los 100 primeros renglones** leídos del `.txt` (recordemos que la clave del `KeyValueSource` corresponde al número de línea):

```
public class First100LinesKeyPredicate implements KeyPredicate<String> {

    @Override
    public boolean evaluate(String key) {
        return Integer.parseInt(key) < 100;
    }

}
```

Hazelcast - Key Predicate

- El `KeyPredicate` se agrega al `Job` de la misma forma que los *mappers* y *reducers* utilizando el método `.keyPredicate`

```
Job<String, String> job = jobTracker.newJob(wordsKeyValueSource);

ICompletableFuture<Map<String, Long>> future = job
        .keyPredicate(new First100LinesKeyPredicate())
        .mapper(new TokenizerMapper())
        .reducer(new WordCountReducerFactory())
        .submit();
```

Combiner

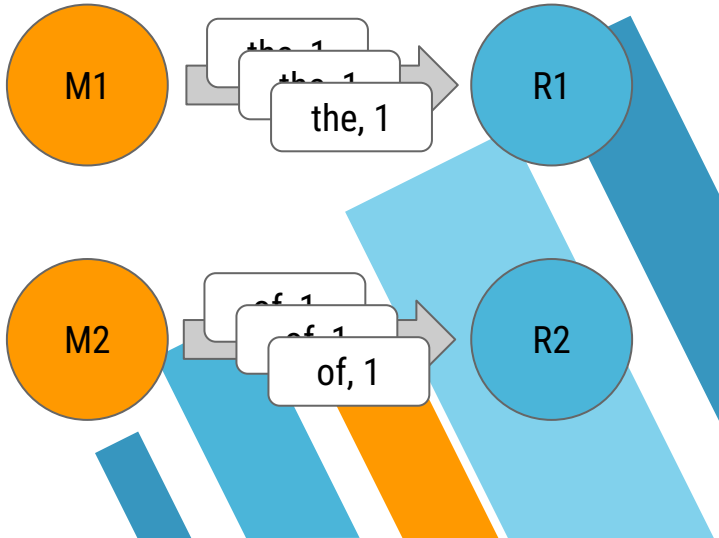
Hazelcast - Combiner

Pensemos el caso donde el texto analizar es significativamente más extenso que el actual

Por ejemplo si contamos con 1 millón de apariciones de la palabra `"the"` y 1 millón de apariciones de la palabra “`of`”, para el **WordCount** implementado y suponiendo un cluster de 4 nodos donde cada uno ejecuta un *mapper* y un *reducer* distinto tenemos que:

- Un *mapper* emite 1 millón de pares `{“the”, 1}`
- Otro *mapper* emite 1 millón de pares `{“of”, 1}`

Esto es **ineficiente porque congestiona la red** dado que están viajando muchos pares similares por la red

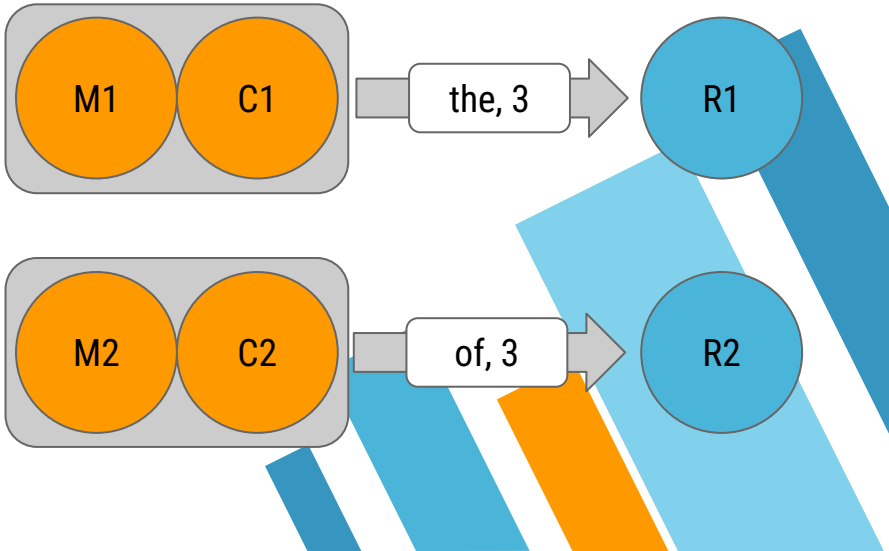


Hazelcast - Combiner

Un `Combiner` funciona como un *reducer* en el mismo nodo del *mapper*, acumulando los valores que salen del mismo antes de emitirlos por la red

Al igual que los *reducer* se instancia uno por clave (y por *mapper*) así que requiere un *factory*

Entonces el tipo de dato del valor de salida del *combiner* debe coincidir con el tipo de dato valor de entrada del *reducer*



Hazelcast - Combiner

```
public interface CombinerFactory<KeyIn, ValueIn, ValueOut>
        extends Serializable {

    Combiner<ValueIn, ValueOut> newCombiner(KeyIn key);

}
```

```
public abstract class Combiner<ValueIn, ValueOut> {

    public void beginCombine() {}

    public abstract void combine(ValueIn value);

    public abstract ValueOut finalizeChunk();

    public void reset() {}

    public void finalizeCombine() {}

}
```

Para implementar un `Combiner` se deben implementar dos clases asociadas:

`CombinerFactory` y `Combiner`

Hazelcast - Combiner

Para el **WordCount**:

```
public class WordCountCombinerFactory implements CombinerFactory<String, Long, Long> {
    @Override
    public Combiner<Long, Long> newCombiner(String key) {
        return new WordCountCombiner();
    }
    private static class WordCountCombiner extends Combiner<Long, Long> {
        private Long sum = 0L;
        @Override
        public void reset() {
            sum = 0L;
        }
        @Override
        public void combine(Long value) {
            sum += value;
        }
        @Override
        public Long finalizeChunk() {
            return sum;
        }
    }
}
```

Hazelcast - Combiner

- A diferencia de los *reducers*, no se espera a recibir todos los valores, sino que **se acumulan hasta que se alcanza una cierta cantidad de valores (*chunk*) y se emite ese resultado parcial**
- El ciclo de vida de un *combiner* es:
  - Al **llegar una clave** que aún no fue asignada a un *combiner* se utiliza **el factory para instanciar al nuevo *combiner***
  - Se llama al método **`beginCombine`** una sola vez (no una por *chunk*)
  - **Por cada valor** asociado a la clave **se invoca al método `combine`**
  - Al acabarse el *chunk* se llama al método **`finalizeChunk`** para obtener el resultado parcial y enviárselo al *reducer*
  - En caso de llegar un nuevo valor se llama al método **`reset`** para inicializar nuevamente el conteo
  - Se llama al el método **`finalizeCombine`** una sola vez (no una por *chunk*)

Hazelcast - Combiner

- El `Combiner` se agrega al `Job` de la misma forma que los *mappers* y *reducers* utilizando el método `.combiner`

```
Job<String, String> job = jobTracker.newJob(wordsKeyValueSource);

ICompletableFuture<Map<String, Long>> future = job
        .mapper(new TokenizerMapper())
        .combiner(new WordCountCombinerFactory())
        .reducer(new WordCountReducerFactory())
        .submit();
```

Collator

Hazelcast - Collator

El `Collator` es una clase que se puede agregar opcionalmente al final del proceso para realizar un *post*-procesado del resultado final del *job*

Recibe un `Iterable` con todas las salidas del proceso, de manera tal que se puede operar tanto con las claves como con los valores de salida

```
public interface Collator<ValueIn, ValueOut> {

   ValueOut collate(Iterable<ValueIn> values);

}
```

Hazelcast - Collator

Para un Job Map Reduce:

- `ValueIn` será `Map.Entry<K, V>`
- `ValueOut` en principio puede ser `Map<K, V>` para que el tipo de dato coincida con el del *job* sin *collator*, pero el usuario puede cambiarlo, pudiendo ser otro tipo de colección o hasta un único objeto

El Collator se ejecutará en el nodo cliente

Hazelcast - Collator

Para el **WordCount** implementado:

- Si queremos ordenar las entradas por clave y valor ya no queremos un mapa (un mapa ordenado sólo tiene orden en las claves) sino una colección ordenada: orden descendente por cantidad de apariciones y desempate alfabéticamente por palabra

```
public class WordCountCollator implements Collator<Map.Entry<String, Long>,
List<Map.Entry<String, Long>>> {

    @Override
    public List<Map.Entry<String, Long>> collate(Iterable<Map.Entry<String, Long>> values) {
        return StreamSupport.stream(values.spliterator(), false)
                .sorted(Map.Entry.<String, Long>comparingByValue().reversed()
                        .thenComparing((Map.Entry.<String, Long>comparingByKey())))
                .collect(Collectors.toList());
    }

}
```

Hazelcast - Collator

- El `Collator` se agrega como parámetro en el `.submit`

```
Job<String, String> job = jobTracker.newJob(wordsKeyValueSource);

ICompletableFuture<List<Map.Entry<String, Long>>> future = job
        .mapper(new TokenizerMapper())
        .reducer(new WordCountReducerFactory())
        .submit(new WordCountCollator());
```

```
sh run-client.sh mobydick.txt
the=55
and=42
of=38
a=25
to=25
...
```

Ejercicio 1

Modificar la implementación del primer

Job Map Reduce Word Count

- Implementar el **KeyPredicate** que sólo considera los primeros 100 renglones del archivo
- Implementar el **Combiner** que reduce el tráfico entre mappers y reducers
- Implementar el **Collator** que ordena los pares de salida
  - Descomentar el `.sorted` que se realizaba en el cliente (porque ya no es necesario)

Alquiler de Bicicletas

Vamos a procesar un dataset de alquileres de bicicletas en la ciudad de Montreal, Canadá (Open data - BIXI Montréal)

Cada línea del archivo CSV representa el alquiler de una bicicleta o viaje, esto es, una bicicleta que se alquiló en una estación y se devolvió en otra estación (puede ser la misma)

Los campos son los siguientes:

- **`startStation`** Estación de inicio del viaje (String)
- **`startDate`** Fecha y hora de inicio del viaje (yyyy-MM-dd HH:mm:ss)
- **`endStation`** Estación de finalización del viaje (String)
- **`endDate`** Fecha y hora de finalización del viaje (yyyy-MM-dd HH:mm:ss)
- **`isMember`** Si el usuario del viaje es miembro

Alquiler de Bicicletas

Ejemplo del archivo `bikeRentals.csv`

```
startStation;startDate;endStation;endDate;isMember
Crescent / René-Lévesque;2022-07-11 03:02:20;Wellington / 2e avenue;2022-07-11 03:32:11;0
Marcil / Sherbrooke;2021-09-15 21:30:11;Harvard / de Monkland;2021-09-15 21:36:24;1
Laurier / de Brébeuf;2021-04-27 12:19:14;Cartier / Masson;2021-04-27 12:21:39;1
Calixa-Lavallée / Sherbrooke;2021-08-03 00:09:53;Villeneuve / de l'Hôtel-de-Ville;2021-08-03 00:23:18;1
Aylwin / Ontario;2022-06-28 18:14:13;St-Urbain / Beaubien;2022-06-28 18:36:04;1
Boyer / St-Zotique;2021-05-08 17:50:26;Boyer / Jean-Talon;2021-05-08 18:00:52;1
```

Se modela una clase `BikeRentalRow` correspondiente a un renglón

Ejercicio 2

- Descargar de Campus el proyecto **hz-bike-rentals**
- Cambiar **l12345** por su legajo para utilizar su propio cluster

Ejecutar el cliente ya implementado con un cluster de al menos dos nodos

El cliente lee el archivo CSV y popula un `MultiMap` donde la clave es la estación de inicio (`String`) y los valores son las instancias de `BikeRentalRow` que tengan a esa estación como inicio del viaje

¿Cuál es la salida obtenida?

Alquiler de Bicicletas

Se obtiene

```
Exception in thread "main"
com.hazelcast.nio.serialization.HazelcastSerializationException:
Failed to serialize 'ar.edu.itba.pod.BikeRentalRow'
```

porque al cargar una instancia de `BikeRentalRow` en una colección distribuida de Hazelcast el objeto viaja por la red y **necesita ser serializable**

Entonces corresponde que `BikeRentalRow` implemente `DataSerializable` y ofrezca un constructor sin parámetros

Alquiler de Bicicletas

```
public class BikeRentalRow implements DataSerializable {
    private String startStation, endStation;
    private LocalDateTime startDate, endDate;
    private boolean isMember;
    ...
    @Override
    public void writeData(ObjectDataOutput out) throws IOException {
        out.writeUTF(startStation);
        out.writeObject(startDate);
        out.writeUTF(endStation);
        out.writeObject(endDate);
        out.writeBoolean(isMember);
    }
    @Override
    public void readData(ObjectDataInput in) throws IOException {
        startStation = in.readUTF();
        startDate = in.readObject();
        endStation = in.readUTF();
        endDate = in.readObject();
        isMember = in.readBoolean();
    }
}
```

Ejercicio 3

Implementar un Job Map Reduce para resolver la consulta Viajes entre estaciones para los usuarios miembros indicando para cada objeto de la salida:

- La estación de inicio
- La estación de finalización
- La cantidad total de viajes entre ambas estaciones realizados por usuarios miembros

usando el record `RentalsBetweenStationsResult` ya provisto

El orden de impresión es descendente por el total de viajes y luego alfabético por nombre de la estación de inicio

Ejercicio 3

Viajes entre estaciones para los usuarios miembros

```
sh run-client.sh bikeRentals.csv
100000
Crescent / de Maisonneuve -> Crescent / de Maisonneuve with 46 trips
Métro Joliette  (Joliette / Hochelaga) -> Aylwin / Ontario with 24 trips
Métro Mont-Royal (Rivard / du Mont-Royal) -> Marquette / du Mont-Royal with 22 trips
Métro Verdun (Willibrord / de Verdun) -> Beatty / de Verdun with 21 trips
Marquette / du Mont-Royal -> Métro Mont-Royal (Rivard / du Mont-Royal) with 20 trips
...
```

Alquiler de Bicicletas

- **Un `Mapper`** que
  - Descarte los `BikeRentalRow` que no correspondan a usuarios miembros
  - Emita { `StationPair`(S1,S2), 1 } de forma que tendremos un *reducer* por cada par de estaciones posible
  - La clase `StationPair` de implementar
    - `DataSerializable` y constructor sin parámetros porque va por la red
    - `equals` y `hashcode` porque debe compararse con otras claves
- **Un `Reducer`** que sume los 1 (Como `WordCountReducer`)
- **Un `Collator`** que retorne un `SortedSet<RentalsBetweenStationsResult>`

Alquiler de Bicicletas

```java
public class StationPair implements DataSerializable {
    private String startStation, endStation;
    ...
    @Override
    public void writeData(ObjectDataOutput out) throws IOException {
        out.writeUTF(startStation);
        out.writeUTF(endStation);
    }
    @Override
    public void readData(ObjectDataInput in) throws IOException { ... }
    @Override
    public boolean equals(Object obj) {
        return obj instanceof StationPair stationpair &&
                startStation.equals(stationpair.startStation) &&
                endStation.equals(stationpair.endStation);
    }
    @Override
    public int hashCode() { ... }
}
```

Alquiler de Bicicletas

```
public class RentalsBetweenStationsMapper
                                implements Mapper<String, BikeRentalRow, StationPair, Long> {

    private static final Long ONE = 1L;

    @Override
    public void map(String key, BikeRentalRow value, Context<StationPair, Long> context) {
        if(value.isMember()) {
            context.emit(
                new StationPair(value.getStartStation(), value.getEndStation()), ONE);
        }
    }

}
```

Alquiler de Bicicletas

```
public class RentalsBetweenStationsCollator
      implements Collator<Map.Entry<StationPair, Long>, SortedSet<RentalsBetweenStationsResult>> {

   @Override
   public SortedSet<RentalsBetweenStationsResult> collate(
                                                  Iterable<Map.Entry<StationPair, Long>> values) {
       SortedSet<RentalsBetweenStationsResult> toReturn = new TreeSet<>(
               Comparator.comparing(RentalsBetweenStationsResult::rentals).reversed()
                       .thenComparing(RentalsBetweenStationsResult::startStation));
       for(Map.Entry<StationPair, Long> entry : values) {
           toReturn.add(new RentalsBetweenStationsResult(
                entry.getKey().startStation(),
                entry.getKey().endStation(),
                entry.getValue().intValue()));
       }
       return toReturn;
   }

}
```

Alquiler de Bicicletas

En el Job Map Reduce se utilizan múltiples instancias de `StationPair` de forma que hay múltiples instancias de `String` correspondientes a los nombres de las estaciones

Podemos reducir el tráfico en la red realizando el Job Map Reduce a partir de los id de las estaciones y luego conseguir el nombre correspondiente al id para las tuplas del resultado final

Para ello podemos usar la interfaz **`HazelcastInstanceAware`** de forma que el mapper pueda consultar otra colección distribuida

```
IMap<String, Station> stationsMap = hazelcastInstance.getMap("stations");
```

Alquiler de Bicicletas

```java
public class Station implements DataSerializable {
    private int id;
    private String name;
    ...
    @Override
    public void writeData(ObjectDataOutput out) throws IOException { ... }
    @Override
    public void readData(ObjectDataInput in) throws IOException { ... }
}
```

```java
public class IdStationPair implements DataSerializable {
    private int startStation, endStation;
    ...
    @Override
    public void writeData(ObjectDataOutput out) throws IOException { ... }
    @Override
    public void readData(ObjectDataInput in) throws IOException { ... }
}
```

Alquiler de Bicicletas

```
public class RentalsBetweenIdStationsMapper
            implements Mapper<String, BikeRentalRow, IdStationPair, Long>, HazelcastInstanceAware {
   private static final Long ONE = 1L;
   private transient HazelcastInstance hazelcastInstance;

   @Override
   public void map(String key, BikeRentalRow value, Context<IdStationPair, Long> context) {
       if(value.isMember()) {
           int startStationId =
hazelcastInstance.<String,Station>getMap("stations").get(value.getStartStation()).getId();
           int endStationId =
hazelcastInstance.<String,Station>getMap("stations").get(value.getEndStation()).getId();
           context.emit(new IdStationPair(startStationId, endStationId), ONE);
       }
   }

   @Override
   public void setHazelcastInstance(HazelcastInstance hazelcastInstance) {
       this.hazelcastInstance = hazelcastInstance;
   }
}
```

Alquiler de Bicicletas

Como el `Collator` se ejecuta en el cliente se puede enviar directamente la colección en el constructor

```
IMap<String, Station> stationsMap = hazelcastInstance.getMap("stations");

Job<String, BikeRentalRow> rentalsBetweenStationsSecondJob =
jobTracker.newJob(rentalsKeyValueSource);

ICompletableFuture<SortedSet<RentalsBetweenStationsResult>>
rentalsBetweenStationsSecondFuture = rentalsBetweenStationsSecondJob
       .mapper(new RentalsBetweenIdStationsMapper())
       .reducer(new RentalsBetweenIdStationsReducerFactory())
       .submit(new RentalsBetweenIdStationsCollator(stationsMap.values()));
```

Takeaways

MapReduce se puede utilizar como alternativa de SQL donde:

- El *Mapper* realiza:
  - ➢ Una proyección equivalente al SELECT
  - ➢ Un filtrado equivalente al WHERE
- El sort/grouping/shuffling equivale al GROUP BY
- El *Reducer* realiza:
  - ➢ Una proyección: idem anterior, pero definitivo
  - ➢ Funciones de agregación
  - ➢ Un filtrado equivalente al HAVING

No son posibles ordenamientos globales de los resultados dentro del modelo de MapReduce ya que el reducer emite por clave

Esto último y alguna *queries* más complejas se hacen mediante un *post* procesamiento o con un segundo MapReduce posterior encadenado a los resultados del primero

Referencias y para profundizar

- Paper MapReduce
- Documentación Hazelcast Map Reduce

CREDITS

Content of the slides:

- POD - ITBA

Images:

- POD - ITBA
- Or obtained from: commons.wikimedia.org

Slides theme credit:

Special thanks to all the people who made and released these awesome resources for free:

- Presentation template by SlidesCarnival
- Photographs by Unsplash